Student Name: Mads Paaske Rasmussen (Madsp24)

Student Exam number: 4140031

GitHub User: Mads-Paaske

GitHub Repo: https://github.com/Mads-Paaske/Component-based-systems

Abstract

As monolithic systems grow the benefits of monolithic architecture start to slip away as the tightly coupled codebase becomes increasingly harder to modify. A solution to this can be found with the Java Platform Module System (JPMS), which allows the application to be split into smaller modules. This modular architecture is the basis of this project, which seeks to replicate the 1979 arcade game Asteroids. The main requirement of the game is the ability to remove modules and run the game without recompilation. This has been achieved using JPMS, the Java ServiceLoader for module discovery and Spring, which allows for inversion of control. Using interfaces, service providers can be identified and used in the core of the game without any need for dependencies between the core and each module. This architecture has made the Asteroids game easier to modify as new components can simply be added without having to change any source code for the core module.

Introduction

Monolithic architecture is the simplest and easiest to develop software architecture until your codebase suddenly starts growing and growing until you have wrapped yourself up in thousands of lines of spaghetti code. The initial benefits such as simplicity of development, testing and deployment quickly turn into growing pains as large tightly coupled systems lack flexibility in terms of development. This means developers have a harder time adapting and evolving a monolithic application as tight couplings means unintended ramifications as all parts of the system depend on each other. Large monolithic systems also become harder to test and deploy as tests for the whole system must be run each time an update is made even if it is a small change [1]. To solve these problems the Java Platform Module System was developed and released in java version 9, which introduced strong encapsulation and explicit dependency management [2]. JPMS resolves the problem of "Classpath Hell", which was a result of the classpath being an unstructured collection of JAR files leading to dependency conflicts, missing classes and version clashes [3].

To understand the benefits of JPMS a component-based replica of the 1979 arcade game Asteroids will be developed. The game is a good candidate for a modular makeover as it features several distinct entities that must interact with each other but don't necessarily share the same capabilities or features. To clone the arcade game, the game must include asteroids slowly drifting towards the player's spaceship. The goal of the game is to shoot these asteroids, which split on impact and reward the player with score. The player tries to get the highest score until they collide with an asteroid and lose the game. Occasionally an enemy spaceship also appears to shoot at the player [4].

Requirements

The goal of this project is to create a component-based version of the 1979 classic, Asteroids. To recreate the game, the elements from the original arcade game, that being asteroids, the player,

enemies, bullets, scorekeeping, and rendering, must be included. To allow components to be added or removed without recompilation the game must be component-based. This means the components must implement interfaces that allow them to provide services to a core module of the game with the responsibility of managing the game's plugins. Each service provider implements an SPI, which defines parameters, return values along with pre and post conditions, that allow the engine running the game to call the service provider without knowing the specific implementation of a given service. This is because implementing an SPI is equivalent to that service provider signing a contract stating that it will provide an implementation of the defined functions. The following requirements are based on the functionalities of the original arcade game as well as the labs for the course.

Functional Requirements

F#	Functional Requirement
F1	The game has an asteroid component
F1.1	The asteroid splits into smaller asteroids when hit
F1.2	The asteroids award score when destroyed
F2	The game has a player component
F2.1	The player can move, which is controlled using the arrow keys
F2.2	The player can use a weapon by pressing the spacebar to destroy asteroids and enemies
F2.3	The player loses the game when colliding with asteroids, the enemy or the enemy's bullets
F3	The game has an enemy component
F3.1	The enemy can move by itself
F3.2	The enemy can use a weapon to attack the player
F3.3	The enemy can be killed by being hit by the player or colliding with an asteroid
F3.4	The enemy awards score when killed by the player
F4	The game has a weapon component
F4.1	The weapon shoots bullets that destroy asteroids, enemies and the player
F4.2	More than one bullet is required to kill the player and enemy
F4.3	The health of the player and enemy is kept track of using a health bar
F4.4	Bullets are destroyed when they go off screen
F5	The game keeps track of the player's score
F5.1	The score is posted when the player dies
F5.2	The game posts scores to a microservice that keeps track of all scores
F5.3	The top 5 scores are viewable
F6	The entities asteroids, player and enemy are rendered
F6.1	The entities asteroids, player and enemy are rendered using different shapes and colors
F6.2	The current score of the player is rendered
F7	The entities asteroids, player and enemy wrap around the screen

Non-Functional Requirements

NF#	Non-Functional Requirement
NF1	Components are removable without recompilation of the program
NF2	The player, enemy and weapon components implement service provided interfaces

NF3	Collision is checked using Pythagoras
NF4	If the microservice is unavailable the game is still playable

Analysis

From the requirements a use case diagram can be created to visualize the requirements of the system from the user's perspective and show relationships between requirements that were not clearly defined in the long list of requirements. The following is a use case diagram for the project based on the requirements from the previous chapter.

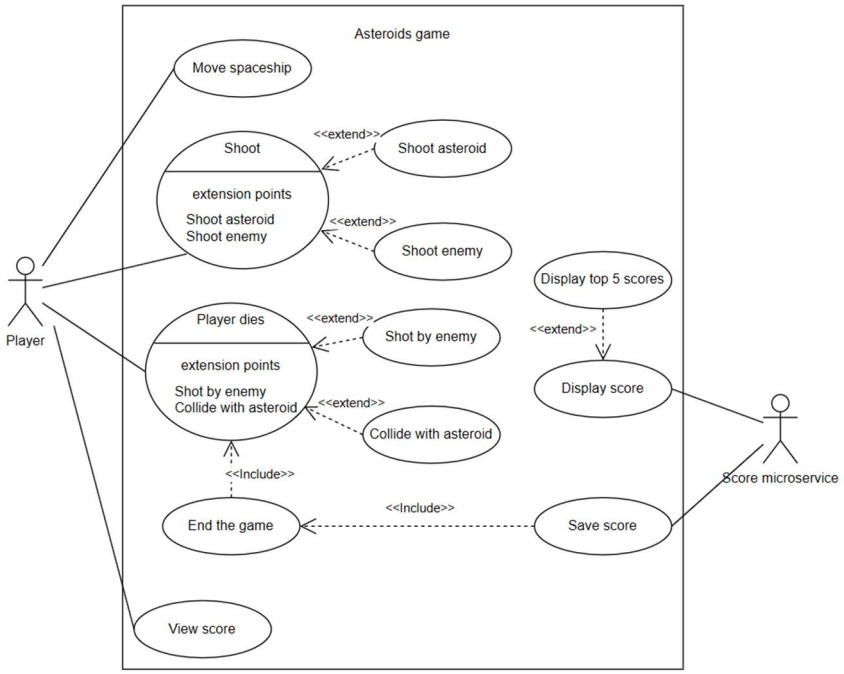

Figure 1: Use case diagram

The use case "Move spaceship" is directly derived from the requirement F2.1, which states the player needs to be able to move. To accommodate this use case the system needs to be able to render entities on the screen for the user to see as well as be able to update the positions of each of the entities on the screen. Specifically for the player, that entity must be controlled by the user using inputs from the keyboard, that the system must read.

The next use case to consider is shooting. The requirements F2.2 and all the F4 requirements relate to this use case. To make sure the player can shoot, bullet entities need to be created and additionally this use case is extended by two other use cases namely shooting an asteroid or shooting the enemy, which is something the player has the option to do when shooting. These use cases are tied to the requirements F1 and F3 and as they both relate to shooting it shows the system must be able to handle collision between entities, but still take into account the differences that exist when collision happens between entities for example the asteroids need to split, which the enemy does not.

The next use case is the player dying, which can happen by either the player being shot by an enemy or colliding with an asteroid, which ties to requirement F2.3. This use case again shows the need for a collision system like the previous use case, but more importantly visualizes what should happen when the game ends. As the diagram shows when the player dies that use case includes the game ending, which includes the other actor in the use case diagram the score microservice. The requirements F5 relate to this scoring service but the use case diagram shows that there needs to be a connection between the player dying and the microservice saving the players score, which facilitates the next use case, viewing score, which is required by F5.3.

From this use case diagram, the main entities of the game are clear as well as the relationships between them. The game needs to feature player, asteroid, enemy and bullet entities, which need to implement collision logic that allows the player to destroy asteroids and enemies, but that also registers when the player dies, which can then end the game. As well as collision logic the entities need to be able to move and be rendered, however like the collision this movement and rendering needs to be distinct to each entity. For example, requirement F2.1 says the player must move with the arrow keys, however the asteroids and the enemy have no such requirement, and they should move autonomously by themselves. The last entity of importance is the microservice, which needs to be able to run externally and save player scores.

From the use case diagram where the entities and their abilities were visualized interfaces allowing for modularity can be defined. The engine running the game must not know the specific implementations of each entity, but instead rely on SPI interfaces, which function as a contract between the engine and the service provider. This is important as modularity is the key non-functional requirement for this project.

To make sure the system fulfills requirements NF1 and NF2 the entities must be modules that function as plugins that can be removed or added without recompilation. That means an IGamePluginService interface responsible for making sure each component implementing it has a start method, which adds the entity to the game and stop method, which removes it. This is required for each entity that needs to be added at the start of the game such as the player, asteroids or the enemy. Furthermore, because these entities need to move an IEntityProcessingService interface must be implemented, which guarantees each module has a processor for their entities. This is important as all the entities must be updated each tick because entities are required to be rendered, move and perform actions every frame. To handle collision, which is required to some degree for all entities an IPostEntityProcessingService interface must be present that checks collision after all other processors are finished. This ensures reliable collision control as the collision check will happen with the most recent position of two entities.

Further interfaces must be implemented to accommodate more specific cases for the entities like a BulletSPI for creating bullets. Because the creation of bullets can happen at any time and not just at the start of the game an interface that all bullet implementations implement allowing bullet creation must be present. Similarly, an asteroid splitting interface AsteroidSplitterSPI must also be implemented to spawn new asteroids, when one is hit by a bullet.

The following is a UML class diagram showing the relationships between the interfaces and entities.

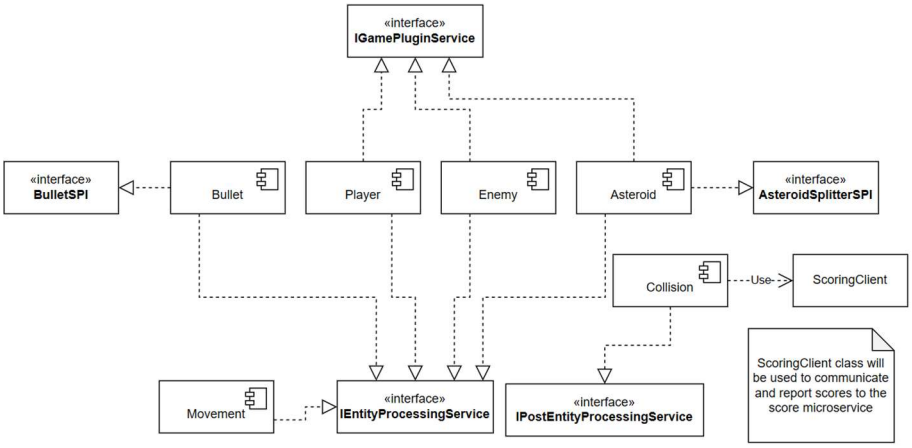

Figure 2: Initial UML class diagram

Now that the components and classes needed for the project have been sketched out considerations about the game flow can be made. There are two main scenarios to consider starting the game and one tick while the game is running. The first sequence diagram is based on the start of the game.

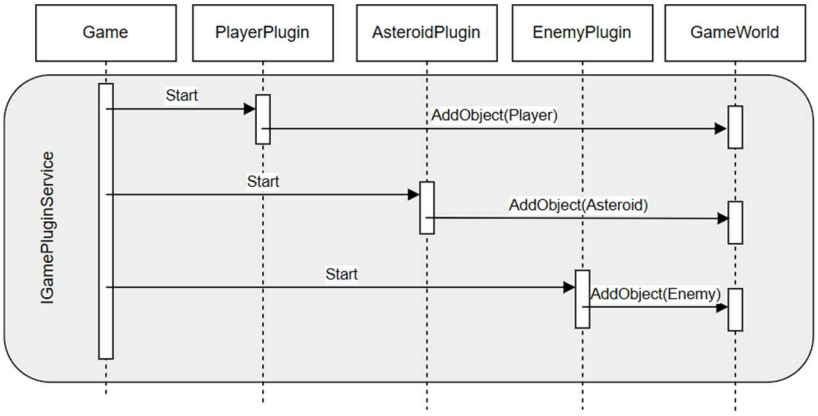

Figure 3: Startup Sequence Diagram

This diagram shows how a central game module responsible for running the game calls the start method defined in IGamePluginService to create each object. This pattern is component-based as the game module has no information about how the different plugins implement the start method, as it is up to the plugins themselves to create the entities. This is also seen in the diagram as game just calls each plugin that then creates the entity itself before adding it to the game world.

The next scenario worth looking at is what should happen during one simulation tick.

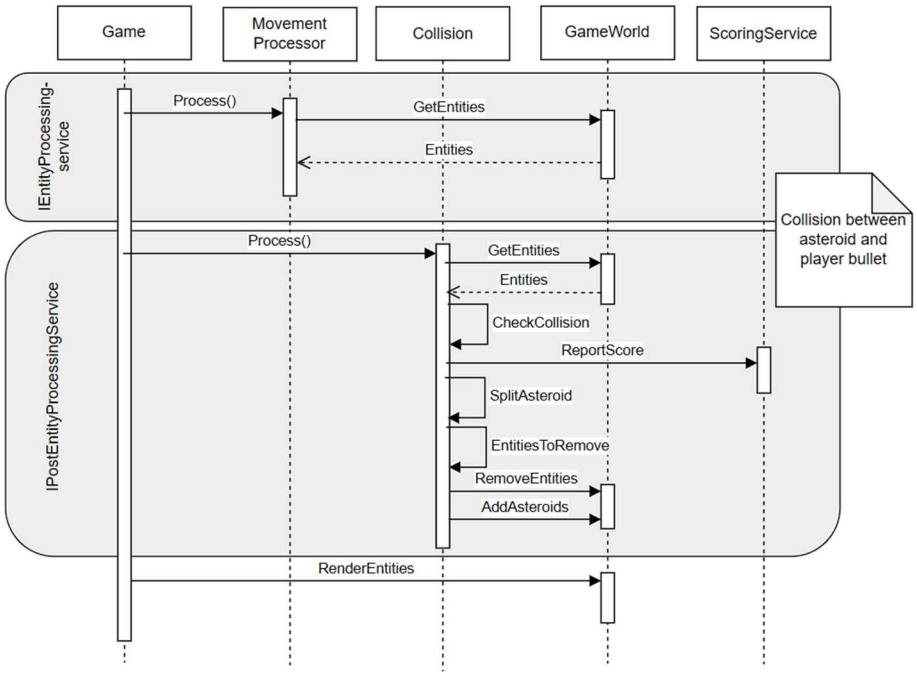

Figure 4: One Tick Sequence Diagram

This sequence diagram shows what interfaces need to be called during one tick of the simulation. In this case the tick shown includes a collision between an asteroid and a bullet fired by the player. First all processers implementing the IEntityProcessingService interface should be called. As the movement processor must implement this interface, the processor is shown getting entities from the GameWorld and updating their positions. After every processor is finished the central Game module calls all the postprocessors through the IPostEntityProcessingService interface, which is the interface, the collision processor must implement. This sequence diagram shows the importance of having separate interfaces for processors and postprocessors as the position of all entities must be updated before collision can be checked.

Design

Now that the entities and interfaces for the game have been sketched out the next step is realizing this sketch into a design that can be implemented. To do this, decisions about the final architecture and technologies used in the game must be made.

The ideal to strive for in component-based architecture is low coupling high cohesion, where low coupling means components must know as little about each other as possible and high cohesion means each component handles one clear responsibility. To achieve this in practice a layered architecture based on the Clean Architecture can be used. An important part of Clean architecture is The Dependency Rule, which states that dependencies can only point inwards [5]. This means the innermost layer of the Clean architecture, that being the entity layer, must not depend on any outer layer, but can be depended upon by other modules. This principle directly facilitates NF1 as

following the dependency rule keeps modules loosely coupled as they only depend on common and not eachother, which is why the project is modelled after the Clean Architecture.

The entity layer should contain the most generic rules for the system, which in this case is classes like GameObject, GameWorld, GameData and Component that are the least likely to be changed. The entire game will be built around these classes, which is why they can be placed in the common module. Following the Clean architecture the next layer is the use cases layer, where all the interfaces for the system live. This layer depends on the entity layer and houses interfaces that allow the system to fulfill its use cases. The interfaces IGamePluginService, IEntityProcessingService, IPostEntityProcessingService, BulletSPI, AsteroidSplitterSPI should be present here allowing for the updates, creations and deletions needed to fulfill each use case specified in the use case diagram. These interfaces could be contained in common modules for specific module types like a module called CommonAsteroid containing the AsteroidSplitterSPI, which would give more control over the dependencies for the system, but due to the relatively small size of the project simplifying the two layers into one module won't have many negative consequences. The next layer worth mentioning is the interface adapter layer, which contains the plugins and processors for the system. Here is where module specific logic is contained and in this case that is specific implementations of methods like start and stop from the IGamePluginService and process from the IEntityProcessingService. Each plugin must implement these interfaces with specific implementations that can be provided to a central game module responsible for starting the game. To follow the dependency rule, these modules must depend on common, with common having no dependencies. The last layer of the clean architecture model is the frameworks and drivers layer. This is where the game module responsible for locating all implementations of certain interfaces will be. The game module is also where rendering will be performed as well as running the game loop.

Alongside the game module there will be an engine module responsible for keeping component definitions that the interface adapters use. This is because the game will use Entity Component System (ECS) architecture, which allows plugins like the player to combine components depending on what capabilities they need. For example, the player can contain a movement component, which tells the movement processor to handle movement for player entities. This is useful because as seen in the analysis different entities need different capabilities like health, which is only needed by the player and enemy. ECS supports Clean architecture as processors don't need to know the specific implementations of entities in the game. Instead they use the components that an entity has to determine whether they should do any processing on the entity. This means new entities can be added and just given a component for the processors to understand that they also must process this entity.

Based on the principles of clean architecture and ECS the following UML component diagram has been created.

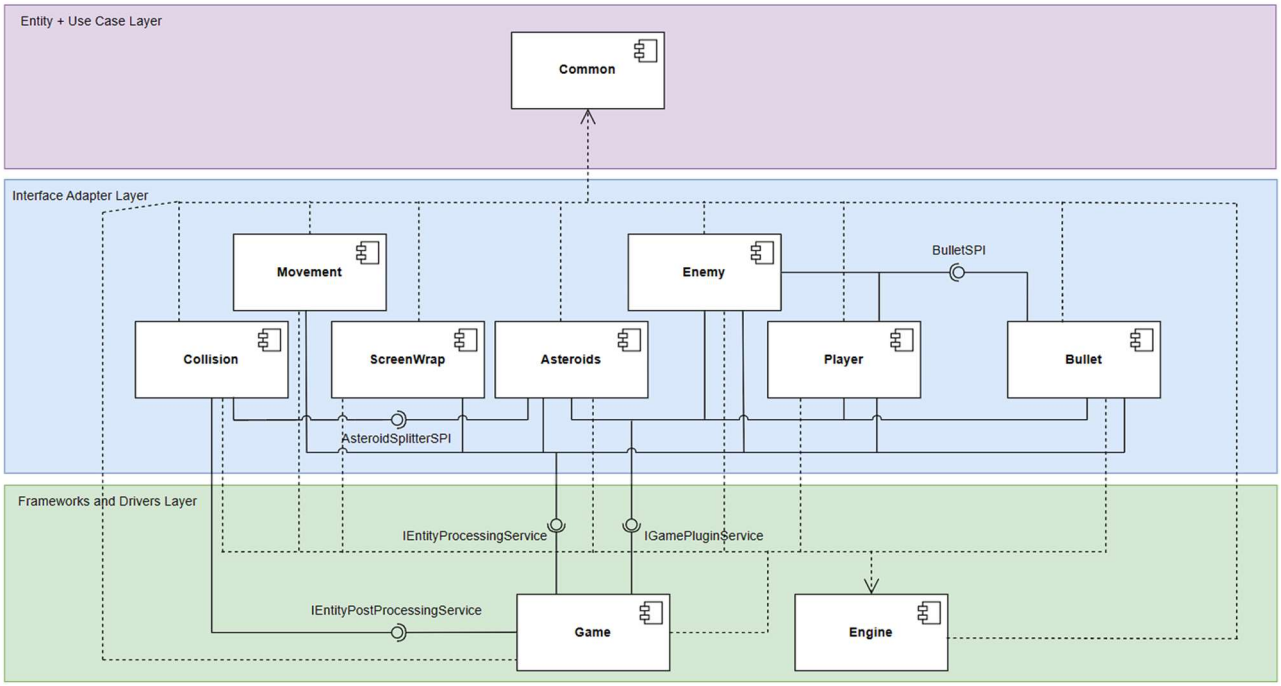

Figure 5: Component Diagram

To make sure the game module can retrieve the entities, processors and post processors needed without glue code, the game module will follow the design principle called inversion of control. Inversion of control will be managed by the Spring Framework IoC container, which will inject the entities, processors and post processors into the game constructor instead of the traditional glue code needed to instantiate each implementation of an interface [6]. The Java ServiceLoader can be used to dynamically discover implementations of interfaces when they are needed. In the game modules with implementations of entities, processors and post processors will be given to Spring to inject at the start of the game. Keeping Spring contained in the game module is another principle of Clean architecture as frameworks are contained in the fewest possible modules meaning if you wanted to change the framework later no changes must be made to modules in the interface adapter layer.

To further specify the interfaces needed for the game component contracts can be made, which draw from knowledge gained from the analysis and design of the architecture.

IGamePluginService		
Description	Needs to be implemented by any plugin that must be instantiated and added to the game world right as the game starts. Plugins implementing this method must also provide a method for stopping the plugin once it has been started.	
Method	Pre-Condition	Post-Condition
Start(GameData, GameWorld)	The game world and game data has been initialized.	The entity exists in the game world.
Stop(GameData, GameWorld)	The method start has been called on the plugin.	The entity has been removed from the game world.

IEntityProcessingService		
Description	Needs to be implemented by all processors so they can process entities using their corresponding component in the game world. The process method will be called every tick to update entities and create new entities if needed like when shooting.	
Method	Pre-Condition	Post-Condition
Process(GameData, GameWorld)	The game world and game data have been initialized. There are entities using the component relating to the processor in the game world.	All related entities are updated, and new entities are added to the game world.

IPostEntityProcessingService		
Description	Needs to be implemented by all processors that need to update entities after the other processors are finished. Mainly used in collision to remove entities.	
Method	Pre-Condition	Post-Condition
Process(GameData, GameWorld)	The game world and game data have been initialized. All processors implementing IEntityProcessingService are done.	All related entities are updated, and new entities are added to the game world.

AsteroidSplitterSPI		
Description	Implemented by asteroids that provide splitting.	
Method	Pre-Condition	Post-Condition
splitAsteroid(GameObject)	The game world and game data have been initialized. An asteroid entity exists and has collided with another entity.	The original asteroid is deleted, and two new asteroid entities are added to the game world.

BulletSPI		
Description	Implemented by weapons that need to spawn bullets for entities that can use weapons. Bullets are spawned with an owner to make sure score is only awarded when a player shoots an asteroid.	
Method	Pre-Condition	Post-Condition
createBullet(GameObject, GameData)	The game world and game data have been initialized. An entity that can shoot exists in the game world and decides to shoot.	A bullet is added to the game world

Implementation

The asteroids game was developed iteratively with each lab building on the next. Each iteration can be found on a branch corresponding to the lab on the GitHub repository for the project [7]. The most important part of the implementation is the game module and common module, as it is these two modules that make the game modular.

Plugins are registered in the ModuleConfig file using Spring and the java ServiceLoader in the game module.

```
@Bean
public List<IGamePluginService> gamePlugins() {
    return ServiceLoader.load(IGamePluginService.class) ServiceLoader<IGamePluginService>
            .stream() Stream<Provider<...>>
            .map(ServiceLoader.Provider::get) Stream<IGamePluginService>
            .toList();
}
```

Figure 6: gamePlugins() in moduleConfig

This code shows an example of the Java ServiceLoader finding all implementations of the IGamePluginService interface. For the ServiceLoader to find an implementation the provider must declare that it provides an implementation of the interface in the module-info file for the plugin, which can be seen in this code snippet.

```
module Player {
    uses dk.sdu.cbse.common.services.BulletSPI;
    requires Common;
    requires Engine;

    provides dk.sdu.cbse.common.services.IGamePluginService
            with dk.sdu.cbse.player.PlayerPlugin;

    provides dk.sdu.cbse.common.services.IEntityProcessingService
            with dk.sdu.cbse.player.PlayerProcessor;
}
```

Figure 7: module-info.java in player module

Because the module-info file in the player module declares that it provides an implementation of the interface the ServiceLoader can find it. This same pattern is used by modules providing implementations of the IEntityProcessingService and IEntityPostProcessingService interface. This supports modularity as it's up to the providers themselves to notify the ServiceLoader instead of the ServiceLoader needing to know the modules themselves.

Because the gamePlugins function is annotated with the @Bean keyword it tells Spring that the list returned by the function must be managed by Spring and injected when needed. This

implementation shows the collaboration between the ServiceLoader and Spring as the ServiceLoader discovers the available plugins for the game, which Spring can then inject into the game's constructor. Compared to the initial implementation of the game module based on the first lab shown here.

```
// create plugin instance
PlayerPlugin playerPlugin = new PlayerPlugin();
playerPlugin.start(gameData, gameWorld);   // adds player to world

AsteroidPlugin asteroidPlugin = new AsteroidPlugin();
asteroidPlugin.start(gameData,gameWorld);

EnemyPlugin enemyPlugin = new EnemyPlugin();
enemyPlugin.start(gameData,gameWorld);
```

Figure 8: First implementation of game module

This pattern achieves inversion of control by replacing the old glue code from game that requires the module to manually declare and start its plugins with the ServiceLoader and Spring that locates and injects the plugins into the game.

The game is implemented using the Java Platform Module System (JPMS), as seen in figure 7. JPMS enforces strong encapsulation by restricting access to packages not declared in the module-info of a module. This strong encapsulation physically enforces Clean architecture. Looking at the player module-info exports is used to make packages accessible to other modules and requires used for declaring a dependency between two modules. This means dependencies must be explicitly stated leading to enforcement of the dependency rule. This also relates to modules providing or using interfaces as modules using an interface must explicitly declare it. The player module uses the BulletSPI and must therefore declare it.

Each plugin is built using the principles of ECS, where each GameObject added to the world is created using components.

```
enemy.addComponent(velocityComponent);
enemy.addComponent(transformComponent);
enemy.addComponent(renderComponent);
enemy.addComponent(new WrapComponent());

enemy.addComponent(new EnemyTag());

gameWorld.addObject(enemy);
```

Figure 9: Adding components to enemy

Each GameObject is created as an entity with components that function as markers for the processors so they can tell which entities they must process. For example, the creation of the enemy

entity includes adding the ScreenWrap component, which is used by the ScreenWrapProcessor to figure out if an entity needs ScreenWrap.

```
@Override
public void process(GameData gameData, GameWorld world) {

    for (GameObject entity : world.getObjects()) {

        if (entity.getComponent(WrapComponent.class) == null) {
            continue;
        }

        TransformComponent t =
                entity.getComponent(TransformComponent.class);
```

Figure 10: Process() in ScreenWrap module

This code shows the process function for ScreenWrap, which either ignores entities without the screenwrap component or gets their transform component to begin to process screen wrapping. Because all entities in the game world are of the type GameObject the processor doesn't care what type of entity is using the ScreenWrap component meaning new modules can use this component without modification to the processors. This pattern is used for all processors implementing the IEntityProcessingService and IPostEntityProcessingService interfaces in the game like movement, collision or processors for the different entities.

As the application requirements state a scoring microservice must be implemented, the project repository also includes a Spring Boot score microservice.

```
@RestController
@RequestMapping("/scores")
public class ScoreController {

    3 usages
    private final List<ScoreRecord> scores = new ArrayList<>();

    Mads-Paaske
    @PostMapping
    public String addScore(@RequestBody ScoreRecord record) {
        scores.add(record);
        return "Score saved";
    }

    Mads-Paaske
    @GetMapping
    public List<ScoreRecord> getScores() { return scores; }

    Mads-Paaske
    @GetMapping("/top")
    public List<ScoreRecord> getTopScores() {
        return scores.stream()
                .sorted(Comparator.comparingInt(ScoreRecord::getScore).reversed())
                .limit( maxSize: 5)
                .toList();
    }
}
```

Figure 11: ScoreController

The annotation @RestController tells Spring that this class will have the ability to handle HTTP requests. The following line shows the @RequestMapping annotation, which defines the base URL for all endpoints for the microservice. The following endpoints allow for Posting scores to the base URL, which can then be viewed either as all scores or the top 5 by going to /scores/top, which can be seen in the @GetMapping annotation of the last function. As this is a simple application no database is connected to the microservice meaning there is no persistence meaning the list storing scores is wiped when the microservice shuts down, which satisfies requirements F5.2 and F5.3, but isn't viable in a real production environment.

```
public class ScoringClient {

    1 usage
    private final RestTemplate restTemplate = new RestTemplate();
    1 usage
    private final String scoringServiceUrl = "http://localhost:8080/scores";

    2 usages  Mads-Paaske
    public void reportScore(String playerName, int points) {
        try {
            HttpHeaders headers = new HttpHeaders();
            headers.setContentType(MediaType.APPLICATION_JSON);

            String body = String.format(
                    "{\"playerName\":\"%s\",\"score\":%d}",
                    playerName, points
            );

            HttpEntity<String> request = new HttpEntity<>(body, headers);
            restTemplate.postForObject(scoringServiceUrl, request, String.class);

        } catch (Exception e) {
            // If the scoring service is down, the game should still work
            System.out.println("Scoring service unavailable: " + e.getMessage());
        }
    }
}
```

Figure 12: ScoringClient

Figure 12 shows the ScoringClient, which exists in the module collision in the project. This class is responsible for posting scores to the microservice using the REST endpoint defined in the microservice. When no microservice is up and running the exception will be caught keeping the game functional even when relying on microservices.

Test

Using Mockito unit tests can be created even when parts of the system like processors depend on other classes keeping the unit test isolated.

```
// Mocks
7 usages
@Mock GameData gameData;
6 usages
@Mock GameWorld gameWorld;
8 usages
@Mock Keys keys;
```

```
// Tell the mocks what to return when the processor calls them
when(gameData.getKeys()).thenReturn(keys);
when(gameData.getDelta()).thenReturn( t: 0.016); // ~60fps frame
when(gameWorld.getObjects()).thenReturn(List.of(player));
```

Figure 13: PlayerTest mocks

Figure 13 shows the code used to mock the required dependencies for the player processor test. In this case mocking the GameWorld, GameData and Keys are required, which can be seen by the @Mock annotations. The values that the objects return can then be defined using the when().thenReturn() function, which in this case is used to make gameData return the mocked keys and a set game delta.

```
@Test
void rotatesRightWhenRightKeyIsDown() {
    when(keys.isDown( key: "LEFT")).thenReturn( t: false);
    when(keys.isDown( key: "RIGHT")).thenReturn( t: true);
    when(keys.isDown( key: "UP")).thenReturn( t: false);

    processor.process(gameData, gameWorld);

    assertTrue( condition: transform.rotation > 0);
}
```

Figure 14: Test for player rotation

Figure 14 shows a test for whether the ship rotates when the right key is held down. Here the when().thenReturn() function is used again to mock return values for the keys object. When the process function is called keys will return true for only the right key making the processor update the ships rotation.

Because the game has been created using the ECS pattern the player entity in this test can be created using only the components needed for the test. This makes the testing simpler as whenever a component needs to be tested it can simply be added to the player object allowing for testing just one component or more depending on the test.

To test whether the game is modular and lives up to its non-functional requirements a video of JAR files being removed from the mods folder of the project can be found with this link.

[https://youtu.be/4qEeUkSjafo]

Discussion

Using the combination of JPMS, the ServiceLoader and Spring, the asteroids game lives up to the non-functional requirements NF1 and NF2 of being able to remove components and still have the

game run without recompilation. While this fulfills the requirements, not all components are removed with the same level of grace. Removing essential plugins like movement leaves the game unplayable as the lack of a movement processor means no entity can move. This highlights a flaw in the design as while the modules can be removed this has no practical application compared to, for example, swapping a weapon or enemy component allowing for varied gameplay.

While the game successfully combined Clean architecture and ECS allowing for entities to be created using reuseable components, which was useful in testing where dependencies could be mocked, and entities built using only the necessary components. The design which required all the modules in the Interface adapter layer to depend on the engine, does violate the dependency rule meaning tighter coupling and reduced modularity. To make the architecture more cohesive the components could be moved to the common module, however this comes at the cost of bloating the common module, which is meant to only hold the most general data and interfaces. In the same vein the common module could also have been split up into different modules for each entity as for example the interface for asteroid splitting and bullet creation should not be known to modules not needing to implement these interfaces.

The application does, however, fulfill all its functional requirements. Asteroids, the player and enemy entities have the capabilities specified in the functional requirements. However, little attention has been paid to the visuals and main game flow as there is no main menu and dying just removes the player from the game, which doesn't end the game. The scoring microservice could also be improved. While it lives up to its requirements the lack of persistent data makes it useless when trying to compare your score with other players.

Conclusion

Setting out with the goal to replicate the classic asteroid arcade game using modular architecture has been a success. After figuring out what was required of the system the use cases and sequence diagrams were used to define essential interfaces and draft design for a modular architecture, which could fulfill the game's most important requirement, namely allowing modules to be removed without recompilation. This design used principles from Clean architecture and ECS to create an architecture where a game module used Spring and the java ServiceLoader to find modules, implementing certain interfaces, using JPMS and inject them into the game's constructor. While improvements could be made to the architecture mainly by splitting the common module and not having modules in the interface adapter layer depend on the engine, the game is modular and lives up to its non-functional requirements. The game shows the viability of modular architecture as the modularity allows for simplified further development as new enemies like bosses or new weapons can be added to improve the replayability of the classic arcade game. However, the most concrete issues for the project involve making the scoring microservice persistent and improving the game loop with main menu and death screens, as well as reevaluating the design of the system to make it follow the dependency rule.

References

[1] O. Dushenin, “Monolithic Architecture: Advantages and Disadvantages,” Medium, Oct. 22, 2021. [Online]. Available: https://datamify.medium.com/monolithic-architecture-advantages-and-disadvantages-e71a603eec89. [Accessed: Jun. 2, 2026].

[2] M. Tyson, “What is JPMS? Introducing the Java Platform Module System,” InfoWorld, Jul. 8, 2020. [Online]. Available: https://www.infoworld.com/article/2257869/what-is-jpms-introducing-the-java-platform-module-system.html. [Accessed: Jun. 2, 2026].

[3] GeeksforGeeks, “JPMS: Java Platform Module System,” GeeksforGeeks, Jul. 12, 2025. [Online]. Available: https://www.geeksforgeeks.org/java/jpms-java-platform-module-system/. [Accessed: Jun. 2, 2026].

[4] The Strong National Museum of Play, “Asteroids,” The Strong National Museum of Play. [Online]. Available: https://www.museumofplay.org/games/asteroids/. [Accessed: Jun. 2, 2026].

[5] R. C. Martin, “The Clean Architecture,” The Clean Code Blog, Aug. 13, 2012. [Online]. Available: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html. [Accessed: Jun. 2, 2026].

[6] Baeldung, “Intro to Inversion of Control and Dependency Injection with Spring,” Baeldung, Apr. 4, 2024. [Online]. Available: https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring. [Accessed: Jun. 2, 2026].

[7] M. P. Rasmussen, “Component-based-systems,” GitHub repository, Jun. 4, 2026. [Online]. Available: https://github.com/Mads-Paaske/Component-based-systems. [Accessed: Jun. 4, 2026].